Calibration-Robust Decentralised Task Allocation for Multi-Agent LLM Systems

Teo Qing Cong Eugene
Independent
www.linkedin.com/in/eugene-teo

July 12, 2026

Technical edition. A plain-language edition with identical results and figures is available as `main.tex`.

Abstract

Multi-agent systems built on a central orchestrator concentrate two costs at one node: it evaluates candidates for every task, and its throughput caps the system's. We study Chemotactic Task Allocation (CTA), a decentralised biomimetic framework in which tasks advertise semantic envelopes and agents self-select when their compatibility clears an activation barrier, with the winner chosen by a cost-adjusted, reliability-weighted Binding Energy and screened by an integrity gate before write access. The framework's load-bearing assumption, that an agent can assess its own fit, is exactly where language-model agents are known to be miscalibrated, so we treat the self-report as a first-class, swept variable rather than an idealisation. Across a pre-registered protocol on synthetic populations, a fleet grounded in measured model calibration, and a live pilot with real coding agents, we find that miscalibration is the failure mode of self-selection and that a track-record correction using no privileged information recovers the completion it costs, with the gate as a safety backstop against adversarial agents. The decentralised design holds peak per-node load flat (fitted growth exponent 0.0) while a central scheduler grows as $N \times M$ (exponent 2.0) out to ten thousand agents, a roughly $250\times$ coordinator cost saving at representative prices. Against a full-information optimum the quality claim does not clear its pre-registered margin and is reported as such; against the information-bounded central scheduler a deployment would actually run, decentralised self-selection matches it with fresh information and overtakes it as the coordinator's reliability table goes stale. Finally, on real Claude agents spanning three model families, the same two mechanisms become a cost result: a task wrapper (an explicit interface contract) lifts a weak model to a frontier model's completion and is the precondition for independently built modules to integrate into a working project, and a calibrated agent wrapper routes to the cheapest capable model, holding frontier-level completion at roughly one fortieth of the always-frontier cost, but only when the tasks are wrapped, so the two wrappers are complementary. Deployed as an Agent Client Protocol broker, the same calibrated router retains almost all of the frontier's completion at a fraction of its cost in a head-to-head against naive-self-report, always-frontier, and single-cheapest routing, and is demonstrated live, end to end, on real agents. All results are deterministic and reproduced from seeds by one command; the real-agent wrappers are powered (the ladder about ten agents per cell, the project five) with bootstrap confidence intervals.

1 Introduction

Multi-agent systems increasingly coordinate large numbers of autonomous agents over many concurrent tasks. A common pattern places a central orchestrator in charge of assigning work from the top down. This pattern is simple to reason about, yet it concentrates two costs at one point: the coordinator must evaluate candidates for every task, and its throughput caps the throughput of the whole system. As the agent and task counts grow, the central node becomes a scaling bottleneck and a single point of failure.

Decentralised alternatives have a long record. The Contract Net Protocol [14] distributes control through an announce, bid, and award exchange, although the award decision for each task still passes through a manager. Market and auction based allocation, surveyed and analysed by Dias et al. [5] and formalised by Gerkey and Matarić [10], casts allocation as an optimisation problem and clarifies where decentralised methods trade optimality for scalability. Stigmergic methods such as ant colony optimisation [7] coordinate indirectly through signals left in a shared environment, with no central assigner at all. More recently, SwarmSys extends the swarm principle to language-model agents, coordinating through pheromone-like traces without central control [12]; the present work differs in adding a trust gate, a calibration correction, and a coordinator-cost scaling result. These results establish that decentralised allocation is viable. Three gaps remain visible across much of this work: the award step is often still centralised per task, a trust or reliability screen before execution is rarely integrated, and there is seldom a principled account of tasks that cannot be done given the agents present.

This work studies a decentralised framework, Chemotactic Task Allocation (CTA), that addresses those three gaps, drawing its selection principle from two biological sources. From cryptic female choice it takes signal-driven, post-advertisement selection: selection continues after the initial encounter and is biased by chemical signalling rather than decided by a single authority [8]. From the response threshold model of division of labour in social insects it takes the activation barrier: an individual engages a task only when the task stimulus exceeds its threshold [2]. Translated to computation: each task is described by a wrapper (the scent envelope) that reads an agent's role, skills, and prompt against the task requirements to produce a compatibility score $c \in [0,1]$. Distributed agents self-select: an agent may take a task only when its compatibility reaches the task's activation energy ($c \geq E_a$), and among the willing agents the winner maximises a cost-adjusted, reliability-weighted score, the Binding Energy $B = c \cdot \tilde{C}/L$. Selection proceeds in two stages, a binary eligibility filter and then the activation barrier, followed by a distinct trust stage, the Rejection Gate. The biology is used as design intuition, not as literal specification.

Compared with a simple pull-based work queue, where agents self-schedule by taking the best available task, CTA adds three things: the activation barrier as a quality floor, the Rejection Gate as a trust boundary, and the eligibility filter with its infeasible and stalled semantics. The evaluation isolates these additions with a pull-based baseline (Section 2.3), so a result is not merely the effect of decentralisation.

Any framework in which agents self-select rests on a load-bearing assumption: that an agent can assess its own fit for a task. Recent studies of auction and bidding among large language model agents report that self-reported success probabilities are systematically miscalibrated, so an allocation built from self-reports diverges from the full-information optimum [9], and calibrating that self-confidence is an open problem [17]. We therefore treat self-assessment as a first-class, swept variable rather than an idealisation. The wrapper's compatibility is the agent's self-report, drawn from its true fit with a bias and noise, while realised quality depends on the true fit and the agent's competence, so miscalibration corrupts the allocation without changing the ground truth. Two questions follow. First, how far does miscalibration degrade a self-selection allocation? Second, does an observable track record (the reliability R), which needs no privileged information, recover the loss? The Rejection Gate then serves as a safety backstop. This repositions the contribution from decentralised self-selection, which is now well covered, onto the calibration robustness of self-selection, which is not.

Cost-aware model selection is an active line: LLM cascades and learned routers send easy queries to a cheap model and hard ones to a strong one, cutting inference cost several-fold without much quality loss [4, 6, 13]. CTA's agent wrapper is a router in this family but differs on three points. First, it corrects the routing signal for miscalibration: the bid is the model's own self-report discounted by an observable track record, rather than a static capability estimate or a preference-trained classifier, so it stays honest when a model is wrong about itself, which Fradkin and Krishnan [9] shows they often are. Second, it routes decomposed work: paired with the task wrapper's interface contract it assembles a single project from modules built by different models, which whole-query routers do not address. Third, it rides a decentralised substrate whose coordination cost stays flat as the fleet grows. Closest to the correction itself, recent work calibrates confidence online for selection: MARGIN learns per-agent, per-band calibration from the task stream for a coordinator deciding which agent to trust [1], and UCCI maps token-level uncertainty to a cost-optimal cascade threshold [11]. Both are centralised; the present contribution is not the online correction on its own but placing it in a decentralised, protocol-native router with an integrity gate against reputation-gaming, and reporting a live head-to-head against these alternatives (Section 4).

Contributions.

1. A decentralised allocation framework in which tasks advertise semantic envelopes and agents self-select by affinity, reducing per-task coordination to an atomic claim rather than a central assignment.
2. A two-stage selection model that separates categorical capability (eligibility) from graded affinity (activation energy), and yields a principled distinction between infeasible tasks (no eligible agent) and stalled tasks (eligible agents present, none clearing E_a).
3. A reliability and integrity trust gate that screens degraded and out-of-scope agents before they gain write access, evaluated as a safety backstop against adversarial agents.
4. An evaluation methodology comparing the framework against centralised baselines on scaling and match quality, including an information-bounded central scheduler as the fair, beatable comparison the full-information optimum cannot provide, and a token/dollar cost model that turns coordinator work into money.
5. A study of self-assessment miscalibration as the failure mode of self-selection: the compatibility bid is treated as the agent's self-report, and a track-record correction (the reliability R), which uses no privileged information, recovers the completion that miscalibration costs, with the integrity gate as a safety backstop.
6. External validity in three steps: a mixed fleet whose calibration archetypes are parameterised from measured LLM behaviour (MarketBench); a live pilot with real Claude coding agents; and a capability ladder across three real model families (Haiku 4.5, Sonnet 5, Opus 4.8) that measures what the task wrapper and agent wrapper buy.
7. A self-improvement result that reads the loop as a harness [15]: a persistent, accumulating track record makes the allocation improve from its own experience toward the full-information optimum with no privileged information, while a memoryless baseline stays flat.
8. A deployed form of the router as an Agent Client Protocol broker [16], evaluated head-to-head against naive-self-report, always-frontier, and single-cheapest routing on real agent outcomes, and demonstrated in a live end-to-end vignette (Section 4).

Scope and non-goals. This is a simulation study with real-agent confirmation, not a live deployment. The atomic claim assumes a single logical coordination store providing compare-and-swap, so the framework reduces central work rather than removing it entirely. The security model covers reliability scoring and scope integrity, and extends to two forms of gaming (bid inflation and reputation sandbagging), addressed in Section 3; cross-agent collusion is left as future work.

2 Methodology

2.1 Research design

The study is a comparative, controlled experiment. Two systems are measured under identical task and agent populations: the decentralised framework and a centralised baseline. The scoring function (Binding Energy) is shared by both systems, so the only difference is how allocation is coordinated. Population size is varied to test the scaling hypothesis, and each configuration is run for many seeded replications to support interval estimates. A simulation mode uses many synthetic agents in Python for scale, determinism, and cheap sweeps; a real-swarm pilot uses Claude Code subagents competing over a shared store (SQLite) where the atomic claim is a genuine compare-and-swap, supplying ecological validity.

2.2 Formal framework

Entities: a set of agents A of size N, and tasks T of size M. Each task carries a scent envelope with a domain, an activation barrier E_a (default 0.2), and a priority. Each agent has a capability profile, permissions, tools, and a reliability history.

Eligibility (binary). $\mathrm{elig}(a,t) = \mathbf{1}[\mathrm{ReqTool}_t \subseteq \mathrm{Tool}_a] \cdot \mathbf{1}[\mathrm{scope}_t \subseteq \mathrm{Perm}_a]$: tools and permitted scope are hard requirements. The eligible set is $A(t) = \{a : \mathrm{elig}(a,t) = 1\}$; the task is *infeasible* when $A(t)$ is empty.

Compatibility and reliability. Compatibility $c(a,t) \in [0,1]$ is produced by the task wrapper from the agent's role, skills, and prompt against the task requirements. Reliability $R(a) = (s_a + 1)/(n_a + 2)$ is a Laplace-smoothed success ratio over a sliding window of W recent attempts. Effective capability is $\tilde{C}(a,t) = C(a,t) \cdot R(a)$.

Selection score. The Binding Energy $B(a,t) = c(a,t) \cdot \tilde{C}(a,t)/\max(L(a,t), \varepsilon)$, with cost $L > 0$ and floor $\varepsilon = 0.01$, ranks the agents that have cleared activation; it does not gate firing.

Activation (firing). Activation drive $\Delta(a,t) = c(a,t) - E_{a,t}$. Firing probability $P_{\mathrm{fire}} = 1$ when $\Delta \geq 0$ and $P_{\mathrm{fire}} = \exp(\Delta/T)$ when $\Delta < 0$, with temperature $T \geq 0$; the deterministic threshold is the $T \to 0$ limit. The task is *stalled* when $A(t)$ is non-empty and the firing set is empty.

Claim and trust. The winner is $a^*(t) = \arg\max_{a \in F(t)} B(a,t)$, committed by an atomic compare-and-swap. The Rejection Gate admits a^* when $R(a^*) \geq \tau$ (default 0.6) and the action is in scope; otherwise it deflects.

Outcome and feedback. Realised quality $Q(a,t) = \mathrm{clip}(g(S_{\mathrm{true}}, C_{\mathrm{true}}) + \xi)$ is a ground truth independent of the agent's self-estimate; the attempt succeeds when $Q \geq q_{\min}$, updating (s_a, n_a) and hence R. Self-assessment acts on noisy estimates $\hat{c} = \mathrm{clip}(c + \eta)$ with $\eta \sim \mathcal{N}(b, \sigma^2)$ for bias b and noise σ; activation and selection use the estimates, outcomes use the truth. Annealing relaxes $E_{a,t} \leftarrow \max(E_{a,\min}, E_{a,t} - \delta)$ per waiting round.

Cost accounting separates total evaluation work $W_{\mathrm{eval}} = \sum_t |A(t)|$ (worst case $O(NM)$) from coordinator work $W_{\mathrm{coord}} = \sum_t |F(t)|$, the claim attempts contending at the shared store.

2.3 Baselines

The centralised baseline is a single scheduler that assigns tasks using the same scores. `central_best` is the full-information quality optimum for CTA's non-exclusive setting: it gives each task the agent maximising expected realised quality (true fit $\times$ true capability) with agent reuse allowed, so it is the fair upper bound

CTA approaches from below (reference for H2). `central_bounded` is the honest scheduler a deployment would run: it allocates from self-reported compatibility and a reliability estimate that lags because a central table is synchronised in batches; a staleness parameter blends current reliability with an out-of-date value (reference for H9). A third, decentralised baseline is a pull-based work queue without the barrier or gate, isolating CTA's specific mechanisms from decentralisation alone.

2.4 Analysis

Results are reported as means with 95% confidence intervals; for bounded, skewed quality and completion metrics the interval is a bootstrap percentile interval. Each headline comparison reports the seeds per arm a power calculation ($\alpha = 0.05$, power 0.8) requires. Scaling growth of coordinator work against N is fitted in log-log space and its exponent reported with a bootstrap CI. Non-parametric tests (Mann-Whitney U) are preferred, with effect sizes alongside p-values. Hypotheses and acceptance margins are fixed in advance; secondary p-values are corrected for multiple comparisons with a Holm-Bonferroni procedure.

2.5 Validity and threats

The primary threat is external validity: a simulation abstracts away the variable latency, cost, and stochastic output quality of live agents, and the miscalibration of an agent scoring itself. This is mitigated at two levels: a fleet parameterised from the stated-versus-realised gaps MarketBench measures, and a live pilot with real Claude agents (Section 3). Reported failure rates for real multi-agent systems are high and driven by specification ambiguity and coordination breakdown a simulation does not reproduce [3]. Generalisability is addressed by re-running the pre-registered hypotheses under a second, structurally different generative family. Reproducibility: code, seeds, and configurations are versioned, and the deterministic path reproduces exactly.

3 Results

These results are generated by one command over the full protocol (20 seeds, agent counts from 50 to 10,000), and the raw per-seed rows behind every aggregate are released as a flat CSV. The results form one arc. The decentralised design removes the coordination bottleneck (H1) and, against the scheduler a deployment would actually run, matches it and overtakes it as its information goes stale (H9), at a deliberate and bounded quality cost against a full-information optimum it does not claim to beat (H2, H6). The load-bearing risk is that self-selection rests on the agent's own self-report, and self-reports are miscalibrated: that miscalibration is the failure mode (H7), a track-record correction using no privileged information recovers what it costs (H8), and an integrity gate is the safety backstop (H4). This mechanism is confirmed on real Claude agents and turned into the product claim: the task wrapper raises a weak model to frontier completion (H11) and the calibrated agent wrapper delivers frontier completion at a fraction of the cost (H12). Run over many rounds the same loop self-improves (H13).

3.1 Hypothesis verdicts

Table 2 records the verdict for each hypothesis. Of eleven pre-registered synthetic hypotheses, nine are supported and two (the quality claims against a full-information optimum, H2 and H6) are reported as honest negatives.

Table 1: **Table 1:** The whole study at a glance.

Layer	**Claim**	**Headline result**
Scaling	H1	Peak per-node load flat (exponent 0.0) vs quadratic central (2.0) to 10,000 agents
Quality	H2, H6	Reaches $\approx 94\%$ of the fair optimum; the shortfall is mostly the deliberate latency trade
Fair coordination	H9	Level with a fresh central scheduler, decisively ahead (0.88 vs 0.64) as its table goes stale
Calibration failure	H7	Raw self-report winners over-predict success by ≈ 0.20 (Brier/ECE ≈ 0.25)
Calibration fix	H8	Track-record correction recovers completion $0.37 \to 0.76$ ($p < 0.001$), no privileged information
Safety	H4	Integrity gate cuts out-of-scope violations by $\approx 90\%$ under adversarial agents
Routing/liveness	H3,H5,H10	Barrier routes every subtask to a correct specialist (1.00 vs 0.25 floor); annealing bounds stall
Task wrapper	H11	Interface contract lifts the weak model $0.950 \to 0.986$ completion (ten agents/cell, tight CIs)
Agent wrapper	H12	Calibrated routing holds frontier completion at $\approx 1/40$ of the always-frontier cost, when wrapped
Self-improvement	H13	With a persistent record, completion climbs $0.36 \to 0.84$ over rounds toward the oracle; raw stays flat

3.2 Adversary robustness (supporting analyses)

The track record demotes an agent that games its bid: a strategic adversary that inflates its self-report while genuinely incompetent, starting with a clean record, is progressively demoted as its realised failures accrue, its win share falling from ≈ 0.22 to near zero over rounds. A harder threat is a *sandbagging* adversary that games the reputation itself: it does honest work to earn a high reliability, then defects. Under a plain cumulative record this exploits a real lag; a recency-weighted reliability window ages the honest record out and collapses the adversary's share the round after it defects, cutting exploitation damage by $\approx 40\%$ (Figure 3). No reactive record can pre-empt the first defection, only bound it: an exposure cap limits how many tasks a single newly-defecting agent can sweep in the round it turns, from ≈ 17 failed wins unthrottled to ≈ 4 at a cap of one (Figure 4). The scope-based integrity gate is the complementary, reputation-independent backstop.

Generalisation of the wrapper lift. To test whether the task-wrapper lift generalises beyond the primary tier, we built a second, structurally independent tier of eight different trap-dense tasks, disjoint from the primary tier, and ran Haiku and Sonnet under the same bare-versus-wrapped design at three agents per cell. Both models solved every task in both conditions (completion 1.00, lift 0.00). This is an honest bound rather than a null: the wrapper repairs specific, real failures (on the primary tier the $+0.036$ Haiku lift came from one genuine failure mode), and where the model is already reliable the contract is a no-op with no regression, a safety net rather than a generic booster.

Generalisability across generators. The population-dependent hypotheses were re-run under a structurally different `latent` family (smooth cosine compatibility, no skill gate). The verdicts are the same under both families (Table 3). The calibration recovery is also robust to how the miscalibration is generated: on a population whose per-agent bias is drawn from the measured MarketBench archetype mixture, the correction still recovers completion by ≈ 0.48.

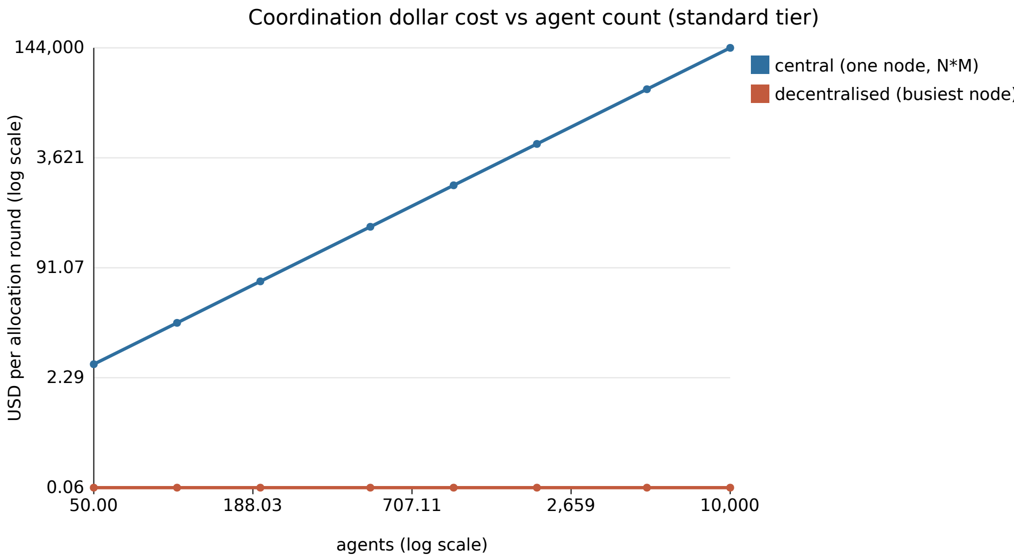

Figure 1: Scaling (H1). The central scheduler's coordination bill grows quadratically in fleet size while the busiest decentralised node stays flat, so the two diverge by orders of magnitude out to ten thousand agents. Prices are representative tiers.

3.3 Real agents: the two wrappers

To observe miscalibration rather than model it, real Claude coding agents stated their confidence on coding micro-tasks with hidden edge-case tests, then solved them one-shot with no tools. Across a nineteen-task suite over two model families (114 attempts), every attempt passed at mean stated confidence ≈ 0.93, so both families were underconfident (overconfidence gap ≈ -0.07), matching the direction MarketBench reports for the Claude models. The sign of the bias is model-dependent, which is exactly why the correction reweights by an observable track record and is agnostic to the direction.

To separate model capability and measure what the wrappers buy, we built a harder expert tier of eight trap-dense tasks and ran three model families (Haiku 4.5, Sonnet 5, Opus 4.8) under a bare prompt and a task-wrapped prompt. Over ten Haiku agents per cell, the task wrapper lifts Haiku from 0.950 (CI $[0.90, 0.988]$) to 0.986 (CI $[0.958, 1.00]$) and leaves the saturated stronger models unchanged (Figure 5). The agent wrapper is Binding-Energy routing across the ladder: over task-wrapped tasks routing holds completion at 0.986 at $\approx 1/47$ of the always-frontier cost; over bare tasks it reaches 0.950 at $\approx 1/50$ (Figure 6). The two wrappers are complementary: the task wrapper raises the weak model's fidelity and the agent wrapper turns that into a cost saving. The ladder rests on ten agents per cell with bootstrap CIs; the project on five.

To test the wrappers on software with decomposition rather than flat tasks, we built *miniquery*, a five-module in-memory query toolkit with a dependency graph and a hidden contract, and had each model family build the whole project under the bare and task-wrapped spec. Under the bare spec every model failed: each made a self-consistent but divergent interface choice, so completion was zero for all three (Figure 7). A calibration signal falls out of the ambiguity: the stronger models correctly lowered their confidence (Sonnet 0.43, Opus 0.62) while the weakest stayed overconfident (Haiku 0.89 on a two-of-five result). The task wrapper, the exact interface contract, made every model deliver a fully conforming project (completion 1.0), and the agent wrapper assembles the cross-model project at $\approx 1/39$ of the always-frontier cost, whereas the bare assembly fails at any price because no model produced a conforming `render` without the contract.

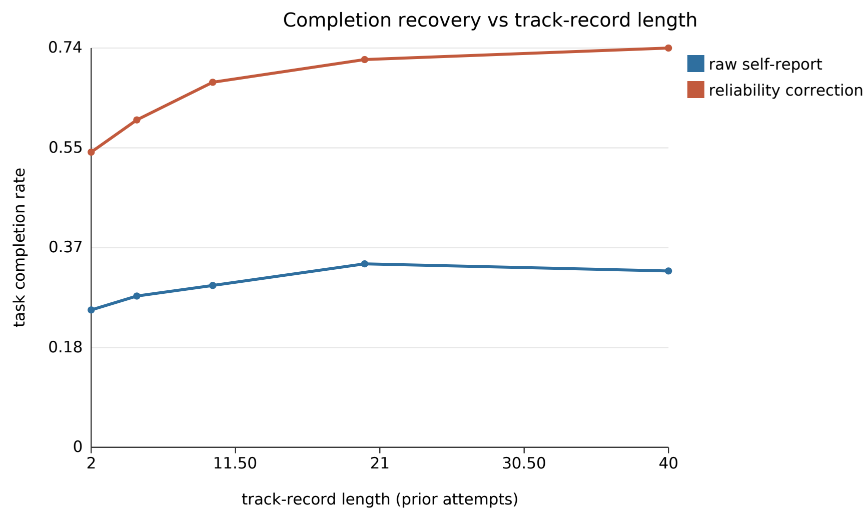

Figure 2: The calibration fix (H8). Discounting each self-report by the agent's observable track record recovers the completion that miscalibration costs, and the recovery needs little history: even a short record recovers most of the gap.

4 Deployment: a calibration-robust routing harness

The agent wrapper deploys as a live router. It is presented as an Agent Client Protocol (ACP) agent [16], a JSON-RPC protocol connecting editors and coding agents. On each turn the broker elicits a confidence bid per candidate model, corrects it by the track record, gates it, routes to the cheapest model that clears the barrier, forwards that model's reply to the editor, and records the outcome. Read as a harness in the sense of Weng [15], the persistent track record is what lets the deployed router improve across sessions rather than start cold each time (the deployment form of H13). Recent work calibrates confidence online for a centralised coordinator or cascade [1, 11]; this router differs in being decentralised, protocol-native, and gated against reputation-gaming.

4.1 Head-to-head against the alternatives

Routing is a decision over outcomes, so it is evaluated on the ladder outcomes already collected, at no extra cost, by leave-one-replicate-out cross-validation: reliability and self-reports are estimated from the training replicates, and each policy is scored on the held-out replicate it did not see. Four policies are compared: CTA-corrected routing, naive self-report routing (the raw bid, no correction), always-frontier, and single-cheapest. Table 4 and Figure 8 give the result on the expert `bare` tier over ten folds. The corrected router retains 0.988 completion, within 0.013 of always-frontier, at $\approx 25\times$ lower cost, and exceeds naive self-report and single-cheapest by $+0.037$. One effect is visible only with real self-reports: because Claude's stated confidence is uniformly high, naive self-report routing collapses into single-cheapest, so the track-record correction is precisely what restores the ability to distinguish hard turns from easy ones.

4.2 A live end-to-end vignette

To exhibit the broker running rather than replayed, it was run live over four real Claude subagent turns, with the track record warm-started from the ladder (Table 5, Figure 9). Three tasks routed to the cheap model (Haiku) and passed. The fourth, `is_match`, is the one task Haiku is unreliable on (reliability 0.70), so its corrected bid falls to 0.595, below the 0.70 barrier, and the broker escalated to Sonnet, which passed. Routed

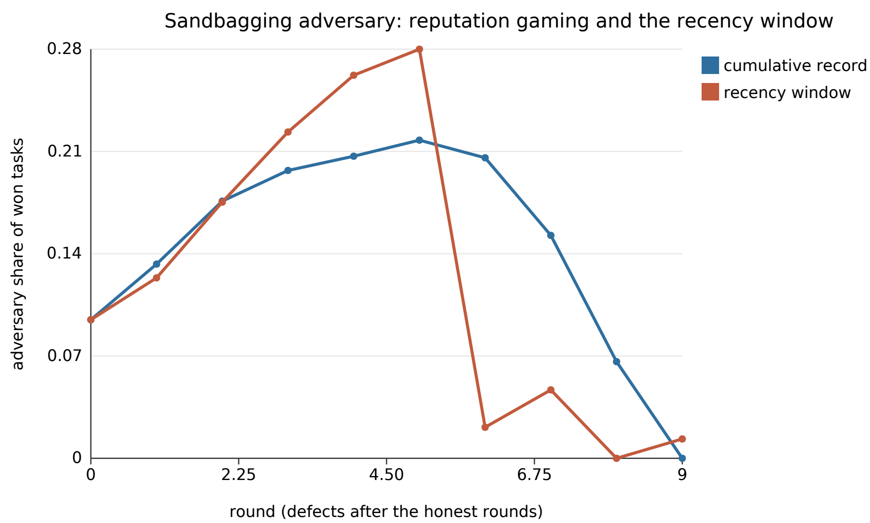

Figure 3: The sandbagging adversary builds a clean record over the honest rounds, then defects. Under a cumulative record its win share coasts down over several rounds; a recency-weighted window collapses it the round after defection.

spend was \$0.045 against \$0.72 for always-frontier, and every turn passed: the broker protected completion where the cheap model was weak and paid the cheap price elsewhere.

5 Discussion

The runs support the scaling and expressiveness claims, and the repositioning around calibration turns the earlier soft spots into results, but the quality claim H2 does not hold against a fair optimum and this is reported as it stands. On scaling, once the bottleneck is measured as peak per-node load and each agent observes a bounded sample of tasks, CTA's peak load is flat in the population size while the central scheduler grows with $N \times M$; a log-log fit recovers the two growth orders cleanly. On quality, against the fair full-information optimum (`central_best`, agent reuse allowed) CTA reaches $\approx 94\%$ of the optimum and is level with pull-based; a decomposition shows about three quarters of the gap is quality the cost-aware bid trades for lower latency by design, and only about a quarter is the residual from ranking on a noisy track record. CTA is not failing to find quality; it is optimising quality per unit cost.

The central result concerns the assumption self-selection rests on. When the compatibility bid is the agent's own self-report and competence varies, the raw self-report auction is competence-blind: completion falls to ≈ 0.37 while winners over-predict their quality by ≈ 0.20 (H7). The correction is cheap and needs no privileged information: discounting the self-report by the observable track record raises completion to ≈ 0.76, a recovery of ≈ 0.39 that is highly significant after multiple-comparison correction (H8). The H6 comparison against a full-information optimum is the wrong test for a deployment, because that optimum cannot exist in production; H9 replaces it with the bounded central scheduler a deployment would run, and CTA is level when its table is fresh and pulls decisively ahead as it goes stale, because a decentralised agent never pays the synchronisation lag a central table does.

On external validity, the miscalibration is not only a synthetic knob: a fleet parameterised from MarketBench reproduces the effect, and a live pilot with real Claude agents confirms on real data that a coding agent's self-reports are miscalibrated. On solvable in-distribution tasks the direction is underconfidence, so a naive auction would under-select capable agents rather than over-select bad ones; the harder tier did not flip the sign, so the overconfident failure region is out-of-distribution here, and populating it needs tasks

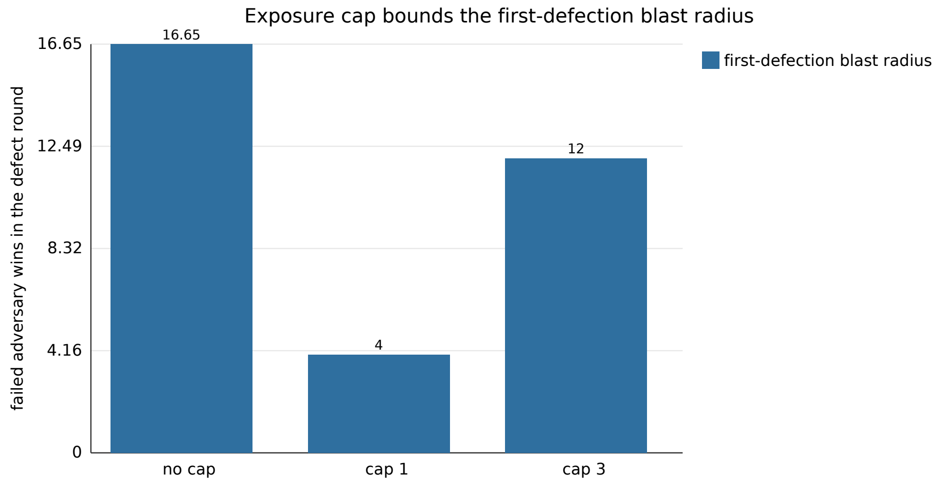

Figure 4: Failed adversary wins in the defection round fall from ≈ 17 with no cap to ≈ 4 at a cap of one task per agent per round. The cap bounds how much a single newly-defecting agent can sweep before the recency window and the gate react, at a coverage cost.

beyond a model's competence or an in-distribution-overconfident model family. This loop is a *harness* in the sense recent work gives the term [15]: the software around a base model that decides how it plans, what it remembers, and how its work is judged. That literature's diagnosis is that a self-improving loop is only as good as the signal it optimises, and the evaluator is the weak link; CTA's track-record correction turns an unreliable self-report into a trustworthy allocation signal, and H13 makes the self-improvement concrete. We claim CTA as the calibration-robust signal layer of such a harness, not a full self-improving system.

The scaling result has a direct commercial reading. Turning the pair-evaluation count into money at representative prices, a central scheduler that scores every agent-task pair pays a bill growing as $N \times M$, while a decentralised fleet caps each agent at a bounded sample, so the busiest node's bill is flat and the total grows only linearly: at the standard tier the central cost per allocation round rises from $\approx \$14$ at a hundred agents to $\approx \$144{,}000$ at ten thousand, whereas the decentralised total is $\approx \$576$ and no single node pays more than $\approx \$0.06$, a $\approx 250\times$ saving at the top of the range. The shape, a quadratic central bill against a flat per-node decentralised one, is a property of the coordination structure.

Practical implications. The immediate application is cost control for multi-model coding assistants and agent platforms, the same setting cost-aware routers already target commercially [4, 6, 13]. The value the correction adds is specific and asymmetric: a cheap model that is confidently wrong is expensive, since a plausible but broken answer costs review, rework, and lost trust well beyond the tokens it saved, and the correction escalates precisely those turns while retaining the cheap price elsewhere. Two properties govern whether it is worth deploying. First, the gain is failure-contingent, growing with how often and how overconfidently the cheap tier fails; the setting that rewards it most is therefore a heterogeneous fleet of small, cheap, and frequently overconfident models rather than already-reliable frontier tiers (the underconfident Claude result reported here sits near the no-regression floor, not the ceiling). Second, the defensible element is not the correction itself, which is simple and appears in concurrent work [1, 11], but the surrounding two: a persistent track record that makes the router cheaper and more reliable the longer it runs, accrued from ordinary use with no privileged information (the self-improvement result, H13); and a protocol-native deployment that integrates into existing editors without bespoke work [16]. The mechanism is thus close to a commodity, while the accumulated calibration data and the deployment surface are where durable value

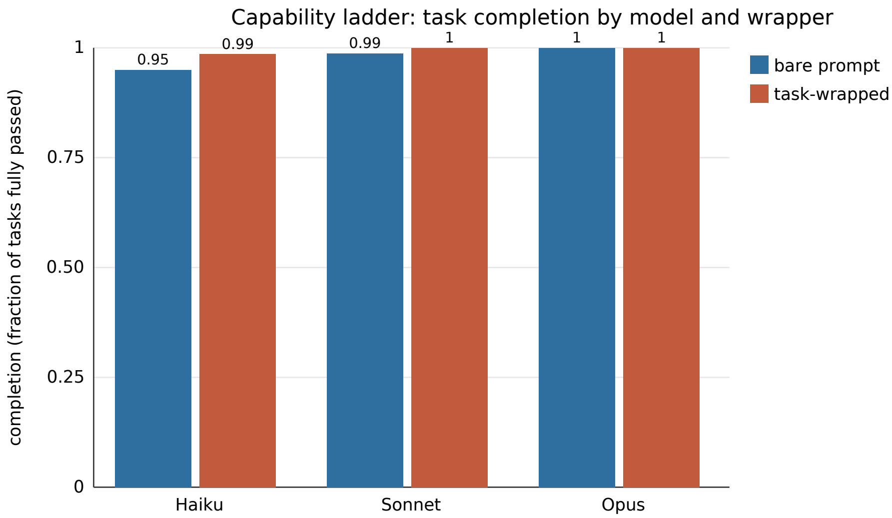

Figure 5: The task wrapper (H11) lifts the weakest model to the frontier's completion and leaves the already-saturated models unchanged. Bars are completion; the wrapper helps most where the model is weakest.

would reside. This is stated as an implication rather than a demonstrated commercial result: the study offers a mechanism with real-agent confirmation, and a production claim would require the head-to-head against a named router on a shared benchmark noted below.

Where this research stands. Eleven pre-registered hypotheses on synthetic populations are evaluated at full protocol scale, nine supported and two reported as honest negatives. Two further hypotheses (the task-wrapper lift and the calibrated-routing cost-efficiency) are added on real Claude agents and supported with bootstrap confidence intervals: the ladder rests on about ten agents per cell and the project on five. It is a mechanism study with real-agent confirmation, not a benchmarked production system, and every claim is marked against that boundary. The natural next step is a head-to-head against a named router on a shared, established benchmark.

6 Building the wrappers

The task wrapper. A task wrapper is an explicit interface contract: a name, a signature, a one-line description, a set of named acceptance criteria, a few worked examples, and a short self-check the agent runs before it commits. It is owned and built by the orchestrator, which makes it stable across a heterogeneous fleet, and it can be constructed from a machine-checkable spec, from examples, from a natural-language description via a cheap wrapping model, or from downstream integration failures added as new acceptance clauses. The design rule is that the acceptance criteria should name the properties a hidden test would check, because the lift comes from surfacing the unstated edge cases that separate a competent-looking answer from a correct one.

The agent wrapper. The agent wrapper turns a fleet of models into a single calibrated router. Each model has a price tier; a fleet holds the models cheapest first and a per-task reliability table. Routing is one rule: each model reports a self-assessed confidence, the bid is discounted by that model's reliability on this task type, and the task goes to the cheapest model whose corrected bid clears the barrier, or escalates to the highest

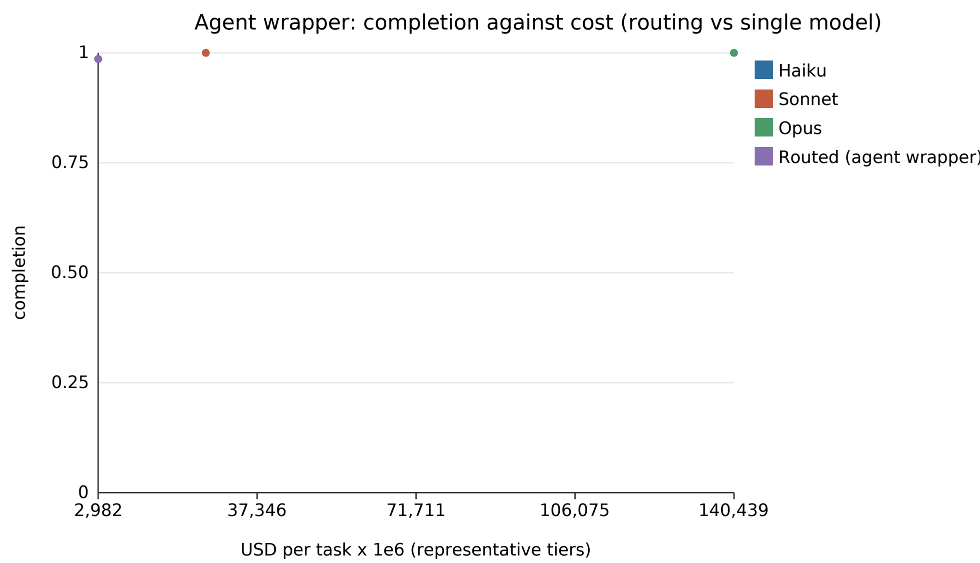

Figure 6: The agent wrapper (H12). Routing over task-wrapped tasks holds frontier completion at a fraction of the always-frontier cost; over bare tasks it leaves the cheap model's residual gap. Cost is on a representative-tier scale.

corrected bid. Reliability is updated after each outcome by a moving average, so the router is competent about the fleet from the first realised results rather than assuming a fixed capability, which is the difference from a static-capability router.

The loop. The two wrappers are not a single call but a loop, and the loop is where the results compound: the track record sharpens, so more tasks safely drop to cheaper models, and the task contracts sharpen from integration failures, so weak models grow more reliable on exactly the tasks that used to break. Calibration robustness and cost efficiency are properties of this loop, not of any one allocation.

7 Outlook and conclusion

The dominant pattern today is a single, large model invoked per task, sometimes fronted by a router that picks a model by static capability. That pattern pays frontier-token prices everywhere and, as the project experiment shows, cannot decompose a task across independent workers without an interface contract. The alternative this work supports is a swarm of microagents: small, cheap, specialised workers, each inside a tight harness, coordinated by calibrated self-selection. The claim to a breakthrough rests on combining three things usually pursued separately: a decentralised substrate that removes the coordinator bottleneck, a routing rule that corrects the self-report by an observable track record and screens the winner for safety, and a task wrapper that is the contract making a swarm's output composable. If the real-agent results are borne out at scale, agent systems can be scaled by the count of cheap microagents rather than the size of one model, a different and far cheaper axis of growth.

In summary, CTA is a decentralised, biomimetic framework in which tasks advertise contracts and agents self-select by a calibrated, reliability-weighted, gated bid. It holds the busiest node's coordination load flat as the fleet grows to ten thousand agents; it names and fixes the failure mode of self-selection (miscalibration), recovering the completion it costs with a track-record correction that uses no privileged information; and on real Claude agents the two wrappers turn the mechanism into a cost result, holding frontier-level completion at roughly one fortieth of the always-frontier cost. The practical thesis, that a swarm of cheap, wrapped

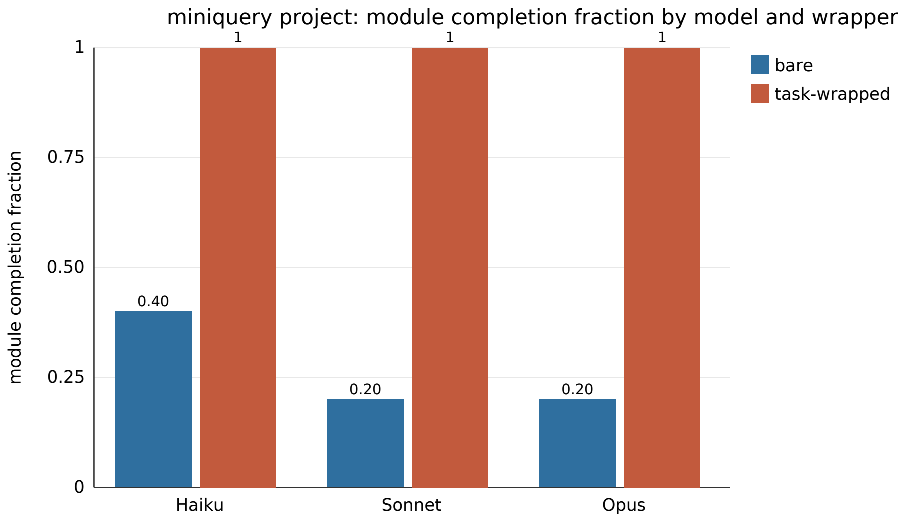

Figure 7: The dependency-graph project. Under a loose spec every model, including the frontier one, ships a divergent interface and no working project; the interface contract makes all three deliver a conforming five-module project. Capability does not rescue integration; the contract does.

microagents coordinated by calibrated self-selection can match a single expensive model at a fraction of its cost, is the direction this work opens.

References

[1] Joss Armstrong. MARGIN: Runtime confidence calibration for multi-agent foundation model coordination. *arXiv preprint arXiv:2605.22949*, 2026.

[2] Eric Bonabeau, Guy Theraulaz, and Jean-Louis Deneubourg. Quantitative study of the fixed threshold model for the regulation of division of labour in insect societies. *Proceedings of the Royal Society of London B: Biological Sciences*, 263(1376):1565–1569, 1996. doi: 10.1098/rspb.1996.0229.

[3] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

[4] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: how to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[5] M. Bernardine Dias, Robert Zlot, Nidhi Kalra, and Anthony Stentz. Market-based multirobot coordination: a survey and analysis. *Proceedings of the IEEE*, 94(7):1257–1270, 2006. doi: 10.1109/JPROC.2006.876939.

[6] Dujian Ding, Ankur Mallick, Shaokun Zhang, Chi Wang, Daniel Madrigal, Mirian Del Carmen Hipolito Garcia, Menglin Xia, Laks V. S. Lakshmanan, Qingyun Wu, and Victor Rühle. BEST-Route: Adaptive LLM routing with test-time optimal compute. *arXiv preprint arXiv:2506.22716*, 2025.

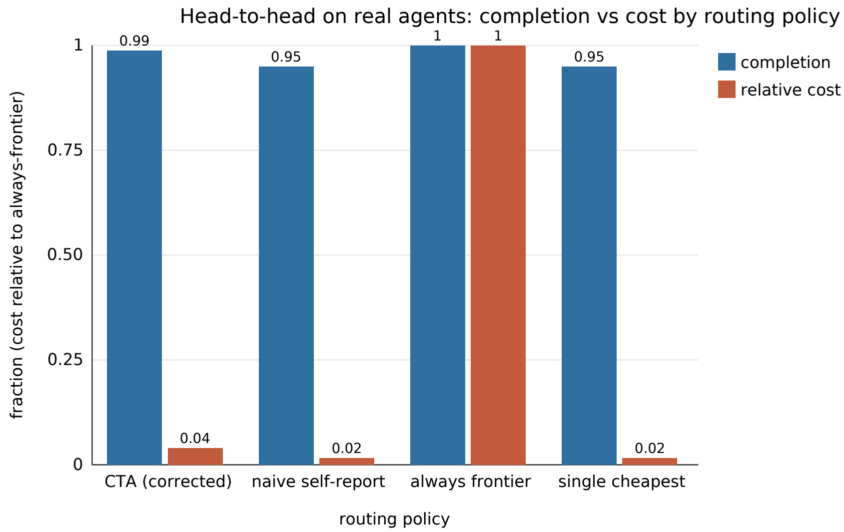

Figure 8: Completion against cost by routing policy, cost as a fraction of the always-frontier cost. The corrected router retains almost all of the frontier's completion at a small fraction of its cost, and separates from naive self-report and single-cheapest, which coincide.

[7] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1):29–41, 1996. doi: 10.1109/3477.484436.

[8] John L. Fitzpatrick, Charlotte Willis, Alessandro Devigili, Amy Young, Michael Carroll, Helen R. Hunter, and Daniel R. Brison. Chemical signals from eggs facilitate cryptic female choice in humans. *Proceedings of the Royal Society B: Biological Sciences*, 287(1928):20200805, 2020. doi: 10.1098/rspb.2020.0805.

[9] Andrey Fradkin and Rohit Krishnan. MarketBench: evaluating AI agents as market participants. *arXiv preprint arXiv:2604.23897*, 2026.

[10] Brian P. Gerkey and Maja J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004. doi: 10.1177/0278364904045564.

[11] Varun Kotte. UCCI: Calibrated uncertainty for cost-optimal LLM cascade routing. *arXiv preprint arXiv:2605.18796*, 2026.

[12] Ruohao Li, Hongjun Liu, Leyi Zhao, Zisu Li, Jiawei Li, Jiajun Jiang, Linning Xu, Chen Zhao, Mingming Fan, and Chen Liang. SwarmSys: Decentralized swarm-inspired agents for scalable and adaptive reasoning. *arXiv preprint arXiv:2510.10047*, 2025.

[13] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[14] Reid G. Smith. The contract net protocol: high-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516.

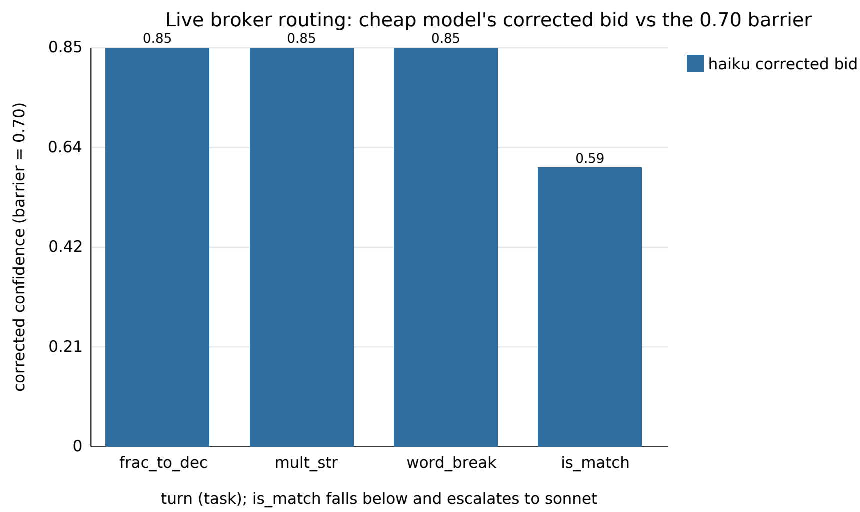

Figure 9: The cheap model's (Haiku) reliability-corrected bid across the four live turns. It clears the 0.70 barrier on the first three tasks, which route to Haiku, and dips to 0.595 on `is_match`, which escalates to Sonnet.

[15] Lilian Weng. Harness engineering for self-improvement. Lil'Log, 2026. https://lilianweng.github.io/posts/2026-07-04-harness/.

[16] Zed Industries. Agent client protocol (acp). https://agentclientprotocol.com, 2025. Open protocol (JSON-RPC 2.0 over stdio) connecting code editors and coding agents; released August 2025, Apache-2.0.

[17] Chao Zhang, Zheng Zhu, Yi Wei, Bo Tian, Jian Liu, Hao Wang, Xin Wang, and Yang Liu. Confidence-calibrated small-large language model collaboration for cost-efficient reasoning. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2026)*, 2026. arXiv:2603.03752.

Table 2: Hypothesis verdicts. "Supported" verdicts carry the observation summarised.

Hypothesis	Verdict	Observation
H1 scaling	supported	CTA peak load flat at 32 as N grows 50 $\rightarrow$ 10,000; central grows as $N \times M$. Log-log exponents 0.0 (CI $[0,0]$) vs 2.0 (CI $[2,2]$).
H2 quality	not supported	CTA reaches $\approx$ 94% of the fair optimum (0.883 vs 0.937), level with pull-based; does not clear the 5% margin. About three quarters of the gap is quality traded for latency by design.
H3 expressiveness	supported	Infeasible and stalled tasks labelled with full recall against ground truth.
H4 safety gate	supported	With $\approx$ 30% adversarial agents and gate recall 0.9, violations cut by $\approx$ 90% (1.2 vs 12.6 per run). A measured reduction, not a tautological zero.
H5 annealing	supported	Without annealing, feasible tasks never claimed; with it, every feasible task resolves at bounded stall (1–3 rounds), falling smoothly with the annealing rate.
H6 heterogeneity	not supported	Against the fair optimum CTA sits slightly below at every heterogeneity; the gap does not close as the population specialises.
H7 miscalibration	supported	Raw self-report winners over-predict realised quality by $\approx$ 0.20 (Brier/ECE $\approx$ 0.25). Structural fit-vs-competence gap, not an artefact of injected bias.
H8 track record recovers	supported	Discounting by R raises completion 0.37 $\rightarrow$ 0.76 under worst overconfidence (recovery 0.39, Holm $p < 0.001$ at 20 seeds).
H9 matches bounded central	supported	Level when the central table is fresh (0.88 vs 0.91); decisively ahead at full staleness (0.88 vs 0.64; advantage 0.25, Holm $p \approx 0.03$).
H10 specialist routing	supported	Barrier holds routing accuracy 1.00 at every observability (chance floor 0.25); without it, routing falls to 0.47 under tight observability.
H11 task-wrapper lift	supported (real)	Over ten Haiku agents/cell, the wrapper lifts Haiku 0.950 (CI $[0.90, 0.988]$) $\rightarrow$ 0.986 (CI $[0.958, 1.00]$); Sonnet/Opus already saturated. On the project every model fails bare (completion 0) and delivers wrapped (completion 1.00).
H12 agent-wrapper cost	supported (real)	Routing over wrapped tasks holds completion 0.986 at $\approx 1/47$ of always-Opus cost; over bare tasks 0.950 at $\approx 1/50$. Project wrapped assembly at $\approx 1/39$; bare fails at any price.
H13 self-improvement	supported	Over twelve rounds, completion under reliability selection climbs 0.358 $\rightarrow$ 0.844, reaching the oracle (0.767), while raw stays flat at $\approx$ 0.35. The gain is the accumulating memory, not the task stream.

Table 3: Verdicts under two structurally different generators.

Hypothesis	Domains family	Latent family
H2 quality within margin	not supported	not supported
H4 safety gate reduces violations	supported	supported
H7 self-reports over-predict	supported	supported
H8 track record recovers	supported	supported

Table 4: Head-to-head on real agents (leave-one-replicate-out replay, expert `bare` tier, ten folds).

Policy	**Completion (95% CI)**	**Cost per task-set (USD)**
CTA (corrected)	0.988 [0.963, 1.000]	0.057
Naive self-report	0.950 [0.913, 0.988]	0.024
Always frontier	1.000 [1.000, 1.000]	1.440
Single cheapest	0.950 [0.913, 0.988]	0.024

Table 5: Live ACP-broker vignette over four real Claude subagent turns. Corrected bids are Haiku/Sonnet/Opus; the cheapest model clearing the 0.70 barrier wins.

Turn	**Task**	**Routed to**	**Corrected bids (h/s/o)**	**Outcome**
1	`fraction_to_decimal`	Haiku	0.85/0.92/0.96	pass
2	`multiply_strings`	Haiku	0.85/0.92/0.96	pass
3	`word_break`	Haiku	0.85/0.92/0.96	pass
4	`is_match`	Sonnet	0.595/0.92/0.96	pass